

AI 应用开发工程师面试准备手册

黄健鸿 | 广州实习 | 跨境阁项目 | 2026 年 9 月 24 日核对版

这份手册围绕你的简历、实际代码和 AI 应用岗位常见能力设计，共 48 道问题，包含参考回答、连续追问和练习。它是有依据的模拟题，不是某家公司的泄露题库，也不能保证面试必问。回答中的“项目现状”来自本次代码核对；“如果扩展”“我会”表示设计方案，不能说成已完成的生产经验。

先掌握 01—08、09—17、23—28，再做 43—48 的现场题。每题先脱稿回答 60 秒，再对照答案补全；涉及原理时，练到能举一个反例，比背名词更有效。

项目事实和表达边界

- 个人情况：仲恺农业工程学院计算机科学与技术，本科在读，2024.09 入学，2028 年预计毕业。无正式实习经历；广州求职，可随时到岗、每周 5 天、可长期实习。
- 项目范围：自 2026.08 起独立负责跨境阁的需求取舍、开发、部署和维护，包含商品推荐、热门商品、知识库问答、图片生成。使用了 AI 编程辅助，要能解释自己负责的设计、检查和排错，不宣称逐行手写。
- RAG 默认参数：目标块长 800 字符、重叠 120 字符；召回 12 个候选，精排最多选 5 个。分块会为元信息前缀预留空间，不能说每块固定 800 字，更不能说 800 token。
- RAG 条件边：检查 documents 是否为空；没有已校准的“资料足够相关”阈值。来源引用与提示规则不能保证零幻觉。
- 持久化：知识库存 Chroma；每日推荐和图片任务使用 SQLite。RAG 图未配置 checkpoint，不能说已经实现 LangGraph 跨进程断点续跑。
- 运行边界：当前是单实例应用；后台线程不是分布式队列；没有已验证的大规模并发、付费客户或模型效果提升百分比。
- 本手册核对了代码和测试定义，未重新运行会调用外部模型或修改业务数据的测试。面试时区分“写了哪些测试”和“某次运行通过了多少测试”。

主要链路：浏览器 → FastAPI → 业务服务 → 模型或数据源 → 持久化 → 页面。RAG 内部为：问题改写 → 向量召回 → 精排 → 整理资料 → 生成或空资料回复。

一 自我介绍与项目深挖

01 用一分钟介绍自己

参考回答： 我叫黄健鸿，是仲恺农业工程学院计算机科学与技术本科生，预计 2028 年毕业，正在寻找广州的 AI 应用开发实习。我独立负责了跨境阁这个全栈项目，主要做了基于 LangGraph 和 Chroma 的知识库问答、公开商品资料推荐，以及 Coze 图片工作流，并用 FastAPI、React 和 Docker 完成部署。我重点实践了模型能力怎样接进真实页面，以及失败时怎样保留数据、避免重复调用和解释当前状态。目前没有企业实习经历，但可以展示代码并讲清设计取舍。可随时到岗，每周五天，可长期实习。

追问： 最能体现你的能力的是哪一部分？优先选 RAG 分支或图片任务恢复，沿输入、状态、失败、测试四步展开，不再报一遍技术栈。

02 这个项目究竟解决什么问题

参考回答： 项目把跨境选品过程里分散的商品资料、知识问答和商品图片生成放在一个应用里。用户能浏览公开商品信息、查看定时生成的推荐、上传知识资料后问答，也能用产品参考图生成图片。我负责把这些流程做成可运行的系统。它当前是个人作品，能证明我具备应用集成与部署实践，不能证明选品建议一定带来销量，也没有商业客户效果数据。推荐里的采样和趋势只是辅助信息，我会保留来源、时间和失败状态，避免把局部资料包装成全市场结论。

追问： 如何判断项目成功？目前看关键流程可用与可恢复；商业阶段还要约定任务完成率、资料时效和用户节省时间，不能用页面好看代替业务价值。

03 为什么不只是调用一次大模型接口

参考回答： 调用模型只完成了一部分。以知识库为例，还要解析文档、去重、分块、向量化、检索与精排，构造来源并处理无资料、超时、流中断。图片生成还存在上传校验、后台状态、供应商链接过期和重复触发费用的问题。我将这些职责拆开，让接口、持久化和前端都有明确契约。项目的价值主要在这条应用链路，而不是我训练了一个模型；我目前没有模型预训练或微调经历。

追问： 换供应商有多难？说明输入、输出、错误和流事件适配层，别说只换一个 URL；图片、鉴权和流格式可能不同。

04 介绍整体架构以及一次问答的路径

参考回答： 前端是 React/Vite，静态资源部署在 GitHub Pages；后端是 FastAPI，经 Nginx HTTPS 反向代理进入 Docker 容器。用户提问后，后端接收问题和有限的历史，通过 LangGraph 先改写检索问题，再到 Chroma 召回、调用精排、组织带编号资料，最后返回答案。SSE 分别传递状态、来源、文本和结束信息。知识库文件、推荐数据和图片数据放在各自的持久化目录。业务复杂度先集中在一个后端里，便于个人项目部署和排错，暂时没有拆成多个微服务。

追问： 画图时哪些是外部依赖？模型、Embedding、Rerank、Coze、商品站点；它们的故障不能统一当作本机服务故障。

05 为什么用 LangGraph 而不用普通函数顺序调用

参考回答： 只有固定两三步时，普通函数已经够用。这个项目有改写失败降级、无资料分支、阶段状态和独立节点测试，用图把状态和下一步选择显式表达，比较容易追踪。节点读取输入状态、返回局部更新，条件边只决定路径。不过图框架不会自动解决检索效果、幂等或持久化；如果为了展示技术而把简单逻辑拆得过碎，也会增加理解成本。我选择它是为了把已有流程和分支说清楚。

追问： 这是自主 Agent 吗？更准确地说是明确步骤的 AI 工作流，不能仅因为用了 LangGraph 就说模型自主规划。工作流与 Agent 的区分可参考 Anthropic 工程文章。

06 图的状态会不会在多个用户之间串用

参考回答： 每次请求应有自己的状态对象，问题、候选片段和最终答案不能放到共享服务实例的可变字段里。项目的图节点从本次 state 读取并返回更新；精排前还复制候选片段，避免它补充分数时原地污染输入。共享的是客户端或服务对象，不等于共享请求内容。我还要检查流式事件回调、上下文变量和取消逻辑的隔离。测试里存在并发请求状态隔离用例，但这不代表已经完成生产高并发压测。

追问： 深拷贝就能解决一切吗？不能，它只处理对象别名问题；共享连接、全局计数器和数据库竞争需要各自的并发控制。

07 你项目中最值得展开的工程取舍

参考回答： 我会选图片任务。供应商可能已经生成并计费，但下载图片时失败。如果把这两步视为一次可以随便重试的请求，就可能重复生成。因此项目用请求标识做提交去重，先保留生成结果的来源，再下载到服务端；保存失败时只重试下载。服务重启后，未完成任务会标记中断，不自动重新触发付费生成。它没有提供分布式恰好一次语义，但把最重要的费用风险和恢复边界区分开了。

追问： 数据库提交前进程崩溃怎么办？承认外部调用与本地事务无法原子提交；未来需供应商幂等键或查询执行结果，再人工核对不确定状态。

08 哪些不足你会优先补上

参考回答： 我会先补小而可信的业务评测集和分阶段观测，再决定优化模型还是检索。当前无资料分支只覆盖空结果，非空但不相关的资料仍可能进入生成；采集也依赖外部站点，不能保证每次成功。部署目前适合单实例，队列和锁不是分布式的。如果面向真实用户扩大使用，接着补权限隔离、费用配额、备份恢复演练和队列化，而不是一开始就引入 Kubernetes。优先级按风险、频率和验证成本排。

追问： 你只有两天怎么办？先选一个关键流程建最小回归集和耗时记录，用真实失败样本验证，不同时改模型、分块和提示词。

二 RAG 与知识库

09 RAG 与微调如何选择

参考回答： RAG 在回答时取外部资料，适合知识经常变化、需要出处或文档权限的场景；微调主要调整模型在特定任务中的行为、格式或风格，需要训练数据和独立评测。跨境阁优先用 RAG，是因为文档可以更新，而且需要让用户看到参考来源。微调不能直接保证最新知识，也不能替代检索权限。如果未来发现稳定的领域表达或结构化输出问题，先优化提示和示例，再比较微调是否值得。两种方式可以结合。

追问： 什么时候不需要 RAG？问题可由确定规则、数据库查询或模型通识稳定解决时，不必额外引入检索链路。

10 Embedding 和 Rerank 分别在做什么

参考回答： Embedding 将文本映射到向量，文档向量可预先保存，查询时较快找出语义接近的候选。Rerank 再对“问题与候选”的相关性重新打分，通常更精细但成本更高，所以只处理小范围候选。项目默认先召回十二条，再最多取五条进上下文。这个组合是当前配置，不是已证明最优。精排无法凭空找回召回阶段漏掉的资料；如果候选里没有答案，应先改数据、分块或召回。

追问： 精排不可用怎么办？当前可退回向量排序并暴露降级状态；不能悄悄宣称仍用了精排。

11 为什么是 800 字符分块和 120 字符重叠

参考回答： 这是项目的默认配置，用于在片段上下文完整性和检索粒度之间取一个起点。实际分块会为标题等前缀预留空间，不是每块固定八百字符。太小会把条件和结论拆开，太大则会混入无关信息；重叠能缓解边界断裂，但也增加重复召回与向量数量。下一步我会用一批带标准证据的问题比较不同块长，而不是把这个数说成行业标准。表格、列表和章节边界应尽量保留。

追问： 字符等于 token 吗？不等，中文、英文、数字与模型分词方式不同；最终还要按实际 tokenizer 估算上下文预算。

12 文件去重为什么需要两次检查

参考回答：项目先对规范化正文做指纹，在分块和调用向量服务之前查重，避免重复处理。两个并发请求可能同时通过预检查，所以持久化阶段还要在锁内重新判断，真正防止重复写入的是后面的保护。正文指纹比文件名更可靠，同一个内容改名仍可识别。现在的锁属于单进程范围，不能直接用于多实例部署。以后需要数据库唯一约束或共享协调，并考虑内容更新时版本怎么替换。

追问：相似但不完全相同的文档呢？SHA256 只判精确规范化结果，不做语义去重；版本和相似检测要另设计。

13 用户说答案不对你怎么定位

参考回答：我先保存脱敏的问题、预期答案和正确证据位置，再逐段检查。先看文档是否成功入库、分块是否保留关键条件；再看改写有没有改变意图、召回候选有没有正确片段、精排有没有把它排掉；最后检查上下文与模型回答。召回里没有证据和有证据但生成错了，修复方式不同。我会固定样本、一次改变一个因素，再验证原失败样本和其他回归样本，不能只反复改提示词到这一题碰巧答对。

追问：用户只说“不好用”？让他给具体任务和期望输出，区分事实错、解释不清、太慢和无法操作。

14 怎样评估 RAG 而不编一个准确率

参考回答：我还没有可以写进简历的完整效果指标。要评估，我会建立包含可回答、无答案、冲突资料和多轮指代的问题集，标注答案要点和证据。检索层看 Recall@k 或命中率，排序看正确证据的排名；答案层看事实支持、引用是否对应和拒答是否合理，再记录延迟与成本。开发集调参数，保留独立测试集验证，避免边看结果边改测试答案。LLM 可以辅助评分，但争议和高风险样本需人工复核。Microsoft RAG 设计与评估指南

追问：Recall@k 怎么算？某题前 k 条里命中的相关证据数除以标注相关证据总数；若每题只标一条证据，可另外报告命中率，别混淆口径。

15 有引用是不是就不会幻觉

参考回答：不是。模型可能引用了真实片段，却把片段含义扩大，或者结论与引用不对应。我的项目要求用资料编号回答，并有空资料分支，但目前 documents 非空就可能进入生成，不能认为资料已经足够回答。改进时我会用不可回答样本校准拒答策略，验证结论与证据的支持关系，并将冲突或缺失明确展示。关键业务结论还应由规则或人工核验。提示词是一个约束层，不是形式化保证。

追问：精排分数高就能直接回答吗？不同模型分数不一定可跨任务比较，更不一定是概率，需要本场景校准。

16 多轮对话改写有什么风险

参考回答： 用户问“那它呢”，检索前需要用历史补全指代，但改写也可能把上文错误带入当前问题，或丢失时间和否定条件。项目传入有限历史做改写，失败时退回原问题。我要保留原问题用于回答和排查，并把改写当作检索辅助，不把历史答案视为新的权威资料。测试应包括主题切换、多个对象、否定句和历史被截断的场景。敏感历史也不能写进普通日志。

追问： 如何做长期记忆？先定义用户同意、保留期限与可删除性，再持久化；当前 conversation_id 并不等于已保存全部会话。

17 换 Embedding 模型为什么可能要重建索引

参考回答： 不同模型生成的向量处在不同语义空间，即使维度恰巧相同，也不能直接把新查询向量和旧文档向量混用。我会为新模型建立新集合，记录模型、维度和分块版本，重新向量化并回归检索效果，再切换查询入口，保留旧版本供回滚。切换期间要保证增量文档也进入目标版本。仅修改模型名称、继续查询旧集合，是一个容易出现静默质量下降的错误。

追问： 新文档必须全量重建吗？同一模型和规则下可以增量；模型或核心解析规则变化时再考虑重建。

18 语义检索和关键词检索如何结合

参考回答： 语义检索适合表达不同但含义接近的问题，关键词检索在型号、编号和专有名词上往往有用。例如精确 ASIN 不能只依赖语义近似。我会先判断是否存在结构化标识，能精确过滤的先过滤；一般文本可以分别召回，再用 RRF 等排名融合方法合并候选并去重，最后精排。不能随意把不同量纲的原始分数相加。当前项目以向量召回与精排为主，混合检索属于后续方案。

追问： RRF 的核心？对各检索结果中的名次取倒数贡献再求和，例如 $1/(\text{常数} + \text{名次})$ ，降低直接比较分数的依赖。

三 模型和 Agent 基础

19 Transformer 和 Attention 需要讲到什么程度

参考回答： Attention 根据 query 与 key 的匹配程度，对 value 做加权汇总，让一个位置能利用上下文信息。缩放点积注意力可写成 $\text{softmax}(QK^T/\sqrt{d})V$ ，除以根号 d 有助于控制点积尺度。Transformer 通过多头注意力、前馈层和位置信息组织序列；生成模型还会用因果掩码避免看到未来 token。应用岗位不必冒充训练专家，但要理解上下文长度、推理成本和信息位置为什么影响效果。原始 Transformer 论文

追问： 大模型是不是数据库？不是，参数中的统计模式不等于可精确更新、可追溯查询的记录系统。

20 Temperature 设为零是否就确定且正确

参考回答： 温度调整 token 分布的尖锐程度，较低温度通常更集中；top_p 是按累计概率限制候选集合。它们影响采样，不验证事实。温度为零也不等于所有供应商和运行环境都会逐字确定，更不等于回答正确。事实问答我会优先控制证据、输出结构和校验，对需要变化的创作再调整采样。比较参数时固定模型版本和问题集，避免把一次回答当成结论。

追问： 为什么不用更大上下文放入所有文档？成本、延迟、噪声和权限边界仍存在；长上下文不替代资料筛选。

21 SFT LoRA 和量化分别解决什么

参考回答： SFT 用有监督样本调整模型行为；LoRA 是参数高效适配方法，通常冻结原权重，训练低秩增量矩阵，从而减少需要训练的参数。量化则降低权重或计算表示精度，主要服务内存和计算开销，也可能影响质量。它们不是同一层的选择。我项目当前调用外部模型，没有做过实际微调训练；如果岗位需要，我会先说明理解范围，再用小规模公开任务验证数据格式、训练和评测流程。LoRA 原论文

追问： 没有优质数据能直接微调吗？不应期待解决全部问题，先查样本质量、覆盖和数据泄漏。

22 Tool Calling 和 MCP 是什么关系

参考回答： Tool Calling 是让模型提出符合约定结构的工具名称和参数，实际执行应由应用代码完成，并进行权限和参数校验。MCP 则提供客户端与服务端连接工具、资源等能力的协议约定，便于接入；它本身不替你决定业务授权和风险。跨境阁目前主要是直接 API 集成与显式工作流，我没有把 MCP 说成项目已上线的模块。若加入工具调用，会限制工具清单、预算和执行次数，有副作用的动作需要明确授权。

MCP 官方架构说明

追问： 模型输出了合法 JSON 就可以执行吗？不行，合法结构之外还要检查租户、对象权限、值范围和幂等。

23 怎样防止 Agent 无限循环和成本失控

参考回答： 我会同时限制总步骤、总耗时、模型 token 预算和工具重试次数，并给每步记录结构化状态。工具失败要区分可重试的短暂错误、需要修正参数的错误，以及必须停止的权限错误。重复调用同一工具却没有新信息时，应停止或请求人工处理。高风险写操作先输出可审查计划，再按授权执行。当前项目采用明确流程，尚没有任意自主工具循环，这是一道扩展设计题。

追问： 是不是多 Agent 更强？不一定，会增加通信成本、错误传播和排查难度；先证明单流程的瓶颈。

四 后端数据与并发

24 async 和多线程有什么区别

参考回答： async 通过等待时让出事件循环提高 I/O 并发，本身不保证 CPU 计算并行。同步网络请求如果直接放进 async 函数仍会阻塞。FastAPI 的普通 def 路由和依赖可在线程池执行，但我在 async 路由里直接调用的普通工具函数不会自动被转移，所以要使用异步客户端或显式线程卸载。线程适合许多阻塞 I/O，CPU 密集计算通常考虑进程或专门服务，还要结合 Python 运行时判断。FastAPI 并发说明

追问： await 前面加什么都能异步吗？不能，被等待对象须支持异步协议，内部也不能长期占用事件循环。

25 Python 常见陷阱与资源管理

参考回答： 我会特别注意可变默认参数、浅拷贝与深拷贝、异常清理以及生成器生命周期。默认参数在函数定义时创建，写 list=[] 可能让多次调用共享列表；应默认 None 后在函数里创建。复制外层 dict 不会复制内部列表，因此精排补元信息时要考虑别名污染。文件、锁和响应流应使用上下文管理或 finally 释放，流式响应正常结束、报错和用户断开都要覆盖。只测正常返回会漏掉连接与并发名额泄漏。

追问： 为什么不能 except Exception 后总返回成功？这样会丢失错误分类并让上层误判，恢复和观测都会失效。

26 为什么使用 SSE 而不是 WebSocket

参考回答： 问答主要是客户端提交一次问题、服务器持续推送文本，单向 SSE 已能满足需求，部署也比较直接。WebSocket 适合频繁双向实时交互，但连接管理更复杂。项目把状态、来源、token 和结束事件分开，前端按事件类型处理。网络的一次 read 不一定是一条完整事件，解析要缓存跨块文本，按空行划分事件，并处理多行 data。Nginx 缓冲也可能导致服务器在流式输出、用户却最后一次才看到。

追问： SSE 一定用 EventSource 吗？不一定，带 POST 请求体时常用 fetch 读取流；结束和重连语义要自行设计。

27 用户停止生成后服务器就不收费了吗

参考回答： 不能保证。浏览器取消请求可以停止继续接收，服务端也应捕获取消并关闭上游流、释放并发名额，但供应商可能已经生成了内容，计费以它的规则为准。如果线程中的同步调用不支持取消，取消外层协程也不代表该调用立即停止。我要把前端停止、服务端任务取消和供应商计费三个层次分开。不能给用户展示“已退回费用”，除非实际有可靠的费用确认接口。

追问： 如何测试清理？用可控的假流在首 token 后断开，检查关闭回调、名额和资源是否释放。

28 SQLite 为什么够用以及什么时候不够

参考回答：目前单实例、短事务和小规模状态数据，SQLite 部署简单，适合保存快照与任务。WAL 允许读写更好地并行，但并不是无限并发写，写入仍要协调。项目用独立连接和短事务，不在事务里等待采集或模型调用。如果出现多个实例、较多并发写、复杂权限或运维需求，我会迁移 PostgreSQL 等服务型数据库。判断依据是压力与维护要求，不是觉得 SQLite 不够高级。SQLite WAL 文档

追问：开启 WAL 还会 busy 吗？会；长事务、写竞争和 checkpoint 等情况仍需处理。

29 事务和幂等为什么不能混为一谈

参考回答：事务保证一组数据库操作按约定一起成功或失败，幂等强调重复请求不会重复产生业务效果。图片任务通过 owner 和 request_id 唯一约束识别相同提交；推荐则按时间槽协调任务。仅仅先查询再插入仍存在并发竞争，必须有数据库约束或事务保护。对于模型供应商，外部调用不在本地数据库事务里，因此还存在“已经执行但我没收到结果”的不确定状态，不能声称用一个锁实现了全链路恰好一次。

追问：同一幂等键却有不同请求体？应比较请求摘要并拒绝冲突，而不是偷偷复用另一个任务；这是扩展时要补的契约。

30 每日推荐如何保证失败时仍有内容

参考回答：项目将任务记录与已发布快照分开。每日按北京时间五点对应的时间槽尝试更新，采集和模型处理在事务外进行，完成后再发布。完整性检查要求十个分类都有商品且分析成功等条件；失败时保留最近可用完整快照，并让页面知道时间与状态。首次没有完整快照时可以保留有商品的部分结果。一次时间槽最多尝试两次，首次失败后有延迟补试，所以不能说它会无限自动重试直到成功。

追问：开多个 worker 可以直接提速吗？不可以，启动恢复会处理 running 状态，当前设计有单实例假设，调度与锁须先改造。

31 为什么采集失败不能归因于一个状态码

参考回答：HTTP 200 只说明拿到了响应，正文可能是验证码、继续购物提示或错误页。我会记录脱敏状态、最终域名、关键结构和商品卡片数，再区分 DNS/超时、重定向、拦截和解析规则变化。商品还要验证 ASIN、标题和价格是否合理。备案主要关联用户访问境内站点的接入要求，不能仅凭它解释服务器向 Amazon 的所有出站失败。当前代码有重试和恢复逻辑，但上游可用性仍需在实际服务器验证。

追问：会不会不断换代理绕过验证码？我不会把绕过访问限制作为可靠交付承诺，必要时选授权数据源、人工导入或降低功能范围。

32 CORS Cookie 和登录校验分别负责什么

参考回答： CORS 决定浏览器是否允许页面读取跨源响应，不是接口鉴权。app 和 api 子域的源不同，仍需正确的 CORS；同站与同源也是不同概念。带 Cookie 的跨源请求需要前端携带凭证，后端允许凭证并明确允许来源，不能随便配星号。Cookie 的 Secure、HttpOnly、SameSite 服务不同目的；真正的对象访问权限必须由服务端检查。项目图片历史的匿名 owner 标识也不等同于正式用户账号体系。MDN CORS

追问： curl 可以访问为何浏览器不行？检查预检、来源、凭证、混合内容和 Cookie；不要先关闭安全限制。

33 前端如何避免旧请求覆盖新状态

参考回答： 用户快速切换分类或会话时，旧请求可能比新请求更晚返回。我会给请求分配序号或使用 AbortController，在写入页面状态前确认它仍属于当前会话；卸载时取消监听和请求。React 状态更新依赖旧值时使用函数式更新，避免闭包读取陈旧值。无限滚动还应防止并发重复取同一页，对稳定商品标识去重，并分别显示初次加载、追加加载、空结果和失败。页面禁用按钮不能替代服务端并发和幂等保护。

追问： 已取消请求还是返回了怎么办？序号校验仍有价值，取消不应作为唯一——一致性措施。

五 安全测试与系统设计

34 文档里面写着忽略规则会发生什么

参考回答： 这是可能的间接提示注入。检索到的内容应被当作资料，不应提升为系统指令；但只在提示词里写“不要听它”并不能保证安全。我会让模型访问的工具遵循最小权限，工具执行前在服务端校验，对写操作增加明确授权，限制可访问的 URL 和数据范围，并用恶意文档做回归。当前知识库主要输出回答，也仍需避免资料诱导泄露内部内容。OWASP 提示注入说明

追问： 做了 RAG 就安全吗？不，检索内容本身也可能成为攻击入口。

35 如何设计多租户知识库

参考回答： 当前个人项目没有企业多租户方案。如果扩展，我会从每个请求的已验证身份得到租户范围，让文档、片段、检索过滤、下载与删除都强制带权限条件，不能接受用户随意传 tenant_id 就直接查询。隔离既可以分集合，也可以共集合加严格过滤，要评估规模、成本和误配置风险。测试要有跨租户负例：A 用户不能通过文档 ID、来源链接或搜索问到 B 的内容。权限过滤应在生成前完成，不能寄希望于模型拒答。

追问： 管理员也能看所有内容吗？由权限模型决定，并记录审计；角色名称本身不是无限授权。

36 你写了哪些有价值的测试

参考回答： 我关注会影响真实用户的分支：空资料时不调用回答模型、改写失败可回退但回答失败不能被掩盖、并发问答不串状态、同文档并发入库只写一次，以及图片保存失败不再次生成。测试通过假模型、假存储或固定响应隔离外部依赖，便于稳定验证。离线测试不能证明供应商今天可用，所以部署后还需要低频真实链路验证。本次求职材料核对的是现有测试定义，我不会凭空报一个最新通过率。

追问： Mock 会掩盖什么？真实协议、鉴权、超时和供应商格式变化，需要少量契约测试与集成验证补足。

37 模型超时或 429 应该怎样处理

参考回答： 先设总时间预算，再分配连接、首包和整体响应超时。对临时故障可有限次退避并加入抖动，尊重供应商 Retry-After；认证错误和确定的参数错误不应盲目重试。读请求和昂贵的生成请求要区别对待，后者先确认幂等或运行状态。如果额度持续不足，应限制入口、说明状态或降级，避免每层都重试形成放大。日志记录供应商请求号、阶段和耗时，不记录完整凭据。

追问： 三层各重试三次会怎样？最坏调用量可能成倍放大，因此需要统一预算与重试归属。

38 把项目扩展为企业知识助手怎么设计

参考回答： 我会先明确用户、文档权限、更新频率和必须回答的问题。第一版划分在线问答和异步入库，文档与版本入数据库，文件进对象存储，任务进队列；解析、分块、向量化有可观察状态。检索先权限过滤，再召回与精排，回答带来源；管理端可查看失败与重试。发布前用约定问题集做效果、延迟和拒答验收。多实例时将本地状态、锁和调度移到共享服务，但不在没有需求和容量依据时提前把所有组件复杂化。

追问： 第一个瓶颈在哪？先测，各系统可能是模型等待、向量检索或文档解析，不能只凭架构图猜。

39 怎样降低延迟和成本

参考回答： 我会拆分改写、Embedding、检索、精排和生成的耗时，分别看首 token 延迟与完整回答延迟。优先去掉无必要调用、控制上下文和输出长度，再考虑较小模型、批量入库和合理缓存。缓存键需要包含问题、权限范围、知识版本与模型配置；不能让不同用户共用不该共享的答案。报告成本时把调用次数、输入输出 token、Embedding 与图片费用分开。每个优化都要回归质量，不能只展示更快。

追问： 有没有提升百分比？目前没有可验证的线上对照数据，只能说设计方向，之后通过固定数据集与运行条件测量。

40 日志监控和隐私怎么兼顾

参考回答： 我希望每次请求能按 request_id 串起阶段、结果类型和耗时，记录召回条数、精排是否降级、供应商错误类别等信息。默认不记录原始密钥、完整文档、用户图片和带签名的下载链接。对排错确实需要的样本，先做脱敏并限制访问与保留时间。当前有基础日志和状态，我不会声称已经部署企业级全链路观测平台。后续应有失败率、P95 延迟、队列积压与费用告警。

追问： 日志全关掉最安全吗？会失去排错和审计能力，应最小化必要记录并控制访问。

六 行为面试与反问

41 项目是 AI 帮你写的你自己会什么

参考回答： 我使用了 AI 辅助写代码、理解报错和整理方案，这是事实。我的责任是提出需求、确认约束、检查修改、运行验证并完成部署。我能解释 RAG 每个节点为何存在、图片为什么区分生成与保存，以及推荐失败为什么保留快照。遇到我暂时解释不了的实现，我不会说已经掌握，会回到代码和文档验证。判断我是否具备能力，可以让我现场修改一个分支、分析失败样本或写一段去重代码，而不只看仓库体量。

追问： 现场能不用 AI 写吗？先完成基本思路与最小实现，不会的库 API 明确说明；不要把生成工具当作回避基础的理由。

42 没有实习经历如何回答协作与职业规划

参考回答： 我目前只有个人项目，不会编造跨部门或客户经历。个人项目里我会先整理问题、确定验收，再把大修改分成可验证步骤，这些方法可以迁移到团队。进入实习后，我希望先负责一个边界明确的接口、检索环节或部署任务，遵守代码评审和发布流程，主动说明风险和进度。我可随时到岗、每周五天、长期实习；具体安排会与课程和公司确认。我的重点是把应用基础做扎实，而不是只追新框架。

可反问： 团队当前最难的是数据质量、模型效果还是上线交付？实习生前三个月负责什么？如何评估问答质量和代码交付？避免只问会用哪些模型。

七 现场编程和数据题

43 保留顺序去重并过滤无效价格

题目：输入商品字典列表，按 ASIN 去重，只保留有限且大于零的数值价格；同一 ASIN 保留第一条有效记录。输入可能有空字段。不要处理无法确定币种的换算，也不要猜测带货币符号字符串的含义。

参考实现：

```
from decimal import Decimal, InvalidOperation

def valid_products(rows):
    seen, result = set(), []
    for row in rows:
        if not isinstance(row, dict):
            continue
        asin = str(row.get("asin") or "").strip().upper()
        raw = row.get("price")
        if not asin or isinstance(raw, bool):
            continue
        try:
            price = Decimal(str(raw))
        except (InvalidOperation, ValueError):
            continue
        if not price.is_finite() or price <= 0 or asin in seen:
            continue
        seen.add(asin)
        result.append(**row, "asin": asin, "price": price)
    return result
```

解释与验收：平均时间 $O(n)$ 、额外空间 $O(n)$ 。先确认有效再加入 seen，否则第一条无效价格会误伤后续有效商品。验证空列表、同 ASIN 大小写、0、负数、None、True、NaN、Infinity，以及“无效记录后出现有效记录”。这里的金额已是上游标准化值；真实项目还要校验 ASIN 格式与币种。

追问：想保留最新记录？先定义更新时间与并列规则，再用字典替换；别把输入顺序默认当时间。

44 限制并发调用并保持输出顺序

题目：异步调用 N 个查询，最多同时执行 3 个，每个在拿到名额后设置 5 秒超时；单个失败要保留对应位置，取消操作要继续传播。

```
import asyncio

async def batch_call(items, call):
    sem = asyncio.Semaphore(3)

    async def one(item):
        async with sem:
            try:
                value = await asyncio.wait_for(call(item), timeout=5)
                return {"ok": True, "value": value}
            except asyncio.TimeoutError:
                return {"ok": False, "error": "timeout"}
            except asyncio.CancelledError:
                raise
            except Exception:
                return {"ok": False, "error": "upstream_error"}

    return await asyncio.gather(*(one(x) for x in items))
```

解释与验收：gather 按提交顺序汇总结果，不按完成顺序。此超时不包括排队等待，底层取消清理也可能使实际返回超过五秒；如题目要求端到端五秒，要把名额等待也放入超时范围。大量 items 仍一次创建很多协程，应改有界队列。依赖 call 必须是可等待且不阻塞的实现。用假调用记录最大并发，验证失败位置、超时和取消。Python asyncio 官方说明

45 用 SQL 查询每个用户最近完成的任务

题目：表 image_jobs 含 id、owner、status、updated_at；每个 owner 取最近完成的一条，并列时取 id 大的。以下为练习表，不是声称项目状态字面值完全一致。

```
WITH ranked AS (
    SELECT id, owner, updated_at,
           ROW_NUMBER() OVER (
               PARTITION BY owner
               ORDER BY updated_at DESC, id DESC
           ) AS rn
    FROM image_jobs
    WHERE status = 'completed'
)
SELECT id, owner, updated_at
FROM ranked
WHERE rn = 1;
```

解释与验收：先筛完成再排名，避免一个用户最新失败任务遮住较早的完成任务。排序字段应可比较且非空，id 用于稳定打破并列。检查无完成任务、相同时间、多个 owner。索引可围绕过滤和排序设计，但选择前看执行计划与写成本，不说“加索引一定更快”。

46 为付费任务设计最小幂等接口

参考回答： POST 接收请求标识，服务端从已验证身份取得 owner，计算请求摘要，在短事务里尝试插入具有唯一约束的任务。相同 owner 和 key 已存在时，摘要一致就返回已有 task_id，不一致就报冲突。外部生成在事务外执行，状态包含 queued、running、succeeded、failed、unknown。若调用超时无法确认供应商执行结果，进入 unknown 查询或人工核对，不能直接再次扣费调用。

测试思路： 并发十次同 key 只出现一条任务；跨用户同 key 不串数据；同 key 不同参数冲突；供应商成功但本地保存失败只补保存；重启不自动重复调用。这个完整契约是设计练习，当前项目已具备其中的请求去重和保存重试，不代表所有 unknown 对账机制均已实现。

47 给知识库设计一个最小评测实验

题目与方案： 准备 30 个练习问题：20 个可回答、5 个无答案、5 个含冲突或条件限制。该数量是训练建议，不是项目已有数据。给每题记录问题、预期要点、证据文档版本、可否回答；将部分题留作最终验证。固定模型和其他设置，分别比较基线与一种改动，例如重排开关。记录命中证据、引用支持、拒答、首 token 与全程耗时，以及输入输出规模。

参考解读： 如果召回提高但回答质量下降，可能混入更多噪声；如果答案更准确但慢很多，需要依据业务目标取舍。少量样本的百分比容易波动，要报告题数与失败类型。最终向面试官展示两三条完整失败分析，比无依据地报“准确率 95%”更可信。

48 一道综合故障题

题目： 前端显示生成失败，日志显示 Coze 已返回四张图片；用户点击重试后产生第二次生成。如何修复与验证？

参考回答： 先定位失败发生在生成、保存还是页面请求响应。如果已有有效来源，保存任务应能独立继续；页面重试应带原任务标识，走保存重试接口，而不是再提交一次生成。服务端要做幂等校验，不能只靠按钮禁用。结果数量、已保存图片与失败项分别展示，避免部分成功被覆盖。对已发生的不确定费用，记录任务和供应商执行标识后核对。回归验证覆盖重复点击、网络断开、部分下载失败与进程重启。

追问： 来源链接已经过期？明确本次无法补存，不能承诺恢复；重新生成应是用户明确发起的新任务。

八 模拟一场 45 分钟面试

- 0—5 分钟：回答 01—03。目标是让面试官知道场景、你的责任和真实边界。
- 5—18 分钟：从 10、12、14、15 连续追问。必须说清召回与生成的不同错误，以及当前不足。
- 18—28 分钟：现场写 43 或 44，先澄清输入和约束，再实现、分析复杂度、举边界案例。
- 28—38 分钟：讨论 29、37、48，看能否区分重试、幂等和外部副作用。
- 38—45 分钟：回答 41—42 并反问。未知问题可以说“我没有做过，但我会先验证……”，随后给具体方法。

自评按四项各 0—2 分：概念准确、结合代码、边界诚实、验证具体。8 分可深入讨论；5—7 分回看薄弱环节；低于 5 分先别背长答案，回到最小代码实验。此评分仅供练习。

九 七天学习安排与证据入口

- 第 1 天：读 RAG 图和配置；画出状态变化，练 01—06、09—12。
- 第 2 天：用五个例题手工对比原问题、改写、召回、精排与答案，练 13—18。
- 第 3 天：补模型与 Agent 基础，练 19—23；所有未做过的内容加上“设计方案”前缀。
- 第 4 天：练 async、SSE、事务和幂等，完成 43—46 的小实验。
- 第 5 天：读异常测试，练 34—40、48；不要为了演示在公开环境反复调用付费生成。
- 第 6 天：录制一次 45 分钟模拟，找出答不出“为什么”和“怎么验证”的地方。
- 第 7 天：只补薄弱项，准备 3 分钟在线演示与离线截图、代码入口备用。

代码阅读路径：backend/zhishiku/zhishiku.py → config_data.py → agent.py → vector_stories.py；再读 backend/tuijian/daily.py、backend/tupian/service.py、provider.py、compose.baota.yaml。代码位置是仓库相对路径，便于用编辑器搜索。

测试证据：backend/zhishiku/tests/test_graphs.py 的空资料、并发隔离、取消释放和入库竞争用例；backend/tupian/tests/test_images.py 的幂等、保存重试与重启用例；backend/tuijian/tests/test_daily.py 的任务和快照用例。

资料补充：LangGraph 持久化说明 checkpoint 与 store 的区别；Docker 启动顺序说明容器运行不等于依赖就绪。这些是原理资料，不能反推你项目已经启用文档中的全部能力。

项目链接：GitHub | 在线作品。本手册不包含 API Key、管理员密码或私有文档。